Performance Engineering

Analysis and Optimisation of the CloverLeaf Mini-App on the HAICGU Kunpeng 920 Cluster

Jort Oosterveld S5545781 & Sergiu-Adrian Preda S6778542
Group 11

Draft, November 4, 2026

1 Introduction

CloverLeaf [2] solves the compressible Euler equations on a 2D structured grid with an explicit Lagrangian–Eulerian scheme; it is a stencil code and a standard case for memory-bound performance engineering [1]. We work on the MPI+OpenMP reference implementation (`CloverLeaf_ref`) and optimise its *C kernels* (`use_c_kernels`) on one node of the HAICGU cluster. Section 2 establishes where the baseline stands against the hardware, Section 3 derives the kernel changes that can still pay off and shows them in code and Section 4 compares the optimised kernels with the baseline at every layout.

2 Baseline Analysis

2.1 Application and system

One time step runs, in order, the equation of state (`ideal_gas`), artificial viscosity, the stable time step (`calc_dt`), a predictor and a corrector PdV step, the node acceleration, the volume fluxes, the advection of cell quantities (`advec_cell`) and of momentum (`advec_mom`) in two directional sweeps and the reset of the time levels. Every kernel is a loop nest over the grid with a ± 1 stencil; the two advection kernels contain an upwind/limiter loop with data-dependent indices. All fields are double-precision arrays of $(n_x + 5) \times (n_y + 5)$ elements. The benchmark problem is **bm3840**: 3840^2 cells, 87 time steps, about 25 arrays of 118 MB ($\approx$3 GB working set, far beyond the 128 MB L3), with CloverLeaf's built-in kinetic-energy check (`PASSED` in every run reported here).

Table 1 lists the node, the software and the run setup. Nodes are used exclusively. A configuration "$r \times t$" is r MPI ranks with t OpenMP threads each. Every rank is bound to t consecutive cores, and OpenMP pins one thread per core. The Fortran kernels are run at the same configurations for reference only.

2.2 Methodology

Tools. Three independent sources: (1) CloverLeaf's internal profiler (wall clock per algorithmic phase); (2) PAPI per kernel, through a small instrumentation layer (`papi_prof`) that wraps the `omp parallel` region of each C kernel, keeps one event set per thread and writes the counts

Node	
CPU	2× Kunpeng 920 (ARMv8), 64 cores each, 2.6 GHz
NUMA	4 domains of 32 cores
Caches	64 KB L1d and 512 KB L2 per core, 32 MB L3 per NUMA domain (128 MB)
Memory	128 GB, 16 channels of DDR4-2666, 341 GB/s theoretical
OS	RHEL-8-compatible Linux, 64 KB pages, 512 MB transparent huge pages
Software	
Compilers	GCC/GFortran 14.1.0
C kernel flags	`-O3 -march=native -funroll-loops -fopenmp` (project `GNU` settings)
MPI	OpenMPI 5.0.3
Measurement	PAPI 7.1.0, Linux `perf`
Runs	
Binding	`-map-by slot:PE=`t, `OMP_PLACES=cores`, `OMP_PROC_BIND=close`
Benchmark	bm3840: 3840^2 cells, 87 steps, about 25 arrays of 118 MB (≈3 GB)

Table 1: System, software and run setup. One node of the HAICGU cluster, used exclusively.

per thread at exit (compiles to nothing without `-DUSE_PAPI`); cycles/instructions, instruction mix and stall sets are measured in separate runs; (3) Linux `perf`: sampling profiles, whole-run counters and the HiSilicon DDR-controller PMU (`hisi_sccl*_ddrc*/flux_rd,wr`), read system-wide around every run, which counts the bytes that cross the memory interface per controller.

Protocol. Five repetitions per configuration, median and min–max reported; the timed region is the complete run of 87 steps including start-up; every repetition runs under the DDR-controller counters, so wall clock and hardware bytes belong to the same run. All bandwidths are *hardware bytes* (DDR-controller counts × 32 B) over wall clock, for STREAM and for the application alike. The harness (`perf/baseline`) is driven by matrix files and environment variables and runs through `mpirun` only.

Checks of the tools. The PAPI build costs nothing measurable (11.30 s against 11.34 s at 64×2; 521–523 s against 528 s on one core) and its per-kernel times add up to the wall clock within 1%; `MEM_ACCESS` equals `PAPI_L1_DCA` in every kernel; the cycle counter gives 2.59 GHz; the DDR counters were calibrated on STREAM (Section 2.4). The placement of every multi-rank run is checked by the counters: the busiest controller group carries 1.00× the mean.

Correctness gate. CloverLeaf's own check on the long deck `clover_bm.in` (960^2 cells, 2955 steps, test problem 3) reports a deviation of 0.0057% (C) and 0.018% (Fortran) from the reference compiled into the code even for the *unmodified* kernels, identical to all digits at every layout and repetition (Table 6 in Section 4): the reference was produced on another machine. Every modified kernel therefore has to reproduce this baseline deviation to 10^{-6}% (`perf/baseline/qa_check.py`) and print `PASSED` on bm3840.

2.3 Where the time goes

Table 2 gives the time share and the counters of every kernel. The two advection kernels take 48% of the time on one core and 63% on 128; `advec_mom` scales worst of the large kernels (14× against 22–59×). On one core the IPC is 0.35–1.07 and 60–87% of the cycles are back-end stalls; at 64×2 the IPC is 0.10–0.50 per kernel and the stall share 81–96%: the cores wait for memory. Three kernel groups appear in the instruction mix: `pdv`, `accelerate` and `flux_calc` are vectorised (41–62% NEON); `calc_dt`, `ideal_gas` and `viscosity` run entirely scalar (they contain `sqrt`, which GCC keeps scalar without `-fno-math-errno`) but are only 12% of the node-level run; `advec_mom` and `advec_cell` are 12–16% vector instructions because their upwind loops

#	kernel	time share (%) 1 core	128 cores	IPC 1 core	vector ins. (%)	FP scalar ins. (%)	back-end stall (%)
1	`advec_mom`	28.6	44.0	0.85	16	17	71 / 94
2	`advec_cell`	19.4	19.4	0.75	12	25	73 / 93
3	`calc_dt`	11.8	4.8	0.65	0	64	75 / 84
4	`pdv_correct`	9.0	5.0	0.35	62	0	83 / 93
5	`pdv_predict`	8.6	4.6	0.35	60	0	87 / 95
6	`ideal_gas`	7.4	4.7	0.52	0	53	79 / 92
7	`viscosity`	5.7	2.2	1.06	0	64	60 / 79
8	`accelerate`	5.2	4.1	0.63	41	0	77 / 93
9	`flux_calc`	2.2	3.0	0.53	45	0	80 / 96
10	`reset_field`	1.1	3.2	0.56	0	0	80 / 97
11	`revert`	0.5	1.3	0.64	0	0	78 / 96
12	`field_summary`	0.4	0.2	0.64	56	24	72 / 90
13	`update_halo`	0.1	3.5	0.67	3	0	49 / 95

Table 2: Per-kernel time share (bm3840; "128 cores" = 1×128), IPC, share of SIMD and scalar floating-point instructions and back-end stall cycles on 1 / 128 cores. Numbers refer to Figure 1. `reset_field`, `revert` and `update_halo` do no floating-point work. Not per kernel: a `perf` profile at 1×128 puts 12.5% of all cycles in the OpenMP barrier (`gomp_team_barrier_wait_end`).

have data-dependent indices. The `perf` profile of a 128-thread run adds that 12.5% of all cycles are spent in the OpenMP barrier (`gomp_team_barrier_wait_end`): every kernel is a sequence of worksharing loops with an implicit barrier each.

2.4 Ceilings and byte model

Memory. STREAM triad (10^8 doubles, 20 iterations, 10 runs per configuration) sustains 15 GB/s on one core, 67 GB/s per NUMA domain (32 threads) and **266.5 GB/s** for the node, in every one of 10 pinned repetitions. Under the DDR-controller counters the controllers move 1.04× the bytes STREAM counts itself (24 B per triad element) and 0.74× the write-allocate model, in all 20 STREAM runs and in our own triad kernel: a streaming store does *not* read the line first on this CPU, one count is 32 B and the hardware ceiling is the counted triad, 78% of the bus. (The DIMM clock is not readable without root; 266.5 GB/s over 16 channels needs at least 2082 MT/s, so DDR4-2666 is consistent.)

Compute. Independent FMA chains on 2-lane NEON registers reach 4.0 flop/cycle per core and **1330 GFLOP/s** per node; scalar FMA reaches the same, so vectorisation saves instructions, not flops. Vector division runs at 0.19 GFLOP/s per core. The ridge point of the node is 1330/266.5 = 5 flop/B (0.7 flop/B for one core).

Byte model. For every loop nest of the C kernels we counted the arrays read and written and the flops per cell; each array is streamed from DRAM once per loop nest, and a store is charged a read-for-ownership (write-allocate) in addition to the write. Table 3 gives 451 flop and 2186 B per cell and step (1688 B without the read-for-ownership), an intensity of 0.21 flop/B.

Validation. The model predicts 2.80 TB for bm3840; the counters measure 2.83 TB at 64×2 in all five repetitions (1% above), 2.79–2.99 TB at the other layouts. The read/write split decides which version of the model applies: the counters see 2.05 TB of reads and 0.77 TB of writes against 1.53 TB of demand reads and 0.64 TB of writes in the model, i.e. the reads are 1.34× the demand reads. Unlike STREAM, the C kernels' stores *do* cause a read of the line first, for about 70% of the written volume; the write-allocate model is the right one for the code as it stands. The Fortran build writes the same amount (0.60 TB) but reads only 1.15× the demand reads

kernel	calls/step	flop/cell	B/cell (no WA)	B/cell (with WA)	AI (flop/B, WA)
`ideal_gas`	2.1	9	32	48	0.19
`viscosity`	1	40	40	48	0.83
`calc_dt`	1	31	80	88	0.35
`pdv_predict`	1	33	96	120	0.28
`pdv_correct`	1	33	112	136	0.24
`revert`	1	0	32	48	0.00
`accelerate`	1	37	80	96	0.39
`flux_calc`	1	10	64	80	0.12
`advec_cell`	2	45	180	244	0.18
`advec_mom`	4	39	172	220	0.18
`reset_field`	1	0	64	96	0.00
`field_summary`	0.1	22	48	48	0.46
per cell and step		451	1688	2186	0.21

Table 3: Analytic flop and byte counts per cell and call (WA = write-allocate). `ideal_gas` runs 2.1 times per step.

kernels	layout	wall (s) median (min–max)	DRAM (TB)	bandwidth (GB/s)	busiest/ mean	vs. hw roof (%)
c	64x2	11.32 (11.24–11.33)	2.83	250	1.00	94
c	1x128	26.91 (17.44–29.20)	2.99	111	1.66	42
c	4x32	13.00 (12.76–13.21)	2.87	221	1.01	83
c	32x4	11.46 (11.41–11.47)	2.79	244	1.00	91
c	128x1	11.66 (11.63–11.77)	2.95	253	1.00	95
c	1x128, THP off	13.31 (13.28–13.92)	2.83	212	1.00	80
fortran	64x2	9.33 (9.33–9.42)	2.35	252	1.00	95
fortran	1x128	20.63 (18.40–25.92)	2.44	118	1.45	44
fortran	4x32	10.88 (10.78–11.22)	2.38	219	1.00	82
fortran	32x4	9.46 (9.44–9.52)	2.30	243	1.00	91
fortran	128x1	9.76 (9.66–9.77)	2.46	252	1.00	94

Table 4: Baseline layouts, bm3840: median of 5 (min–max), hardware bytes of the DDR-controller counters, the resulting bandwidth and its share of the 266.5 GB/s ceiling and the load of the busiest controller group relative to the mean. THP as installed except for the one marked row.

(1.76 TB, 2.36 TB in total): its loops avoid most of that traffic, and this 0.47 TB is the entire C/Fortran gap (Section 2.5).

2.5 Baseline performance

Table 4 gives the layouts of the C kernels with the hardware bandwidth they sustain over the whole run, start-up included. **64×2 is the baseline**: 11.3 s (spread 0.8%), 2.83 TB, 250 GB/s = **94% of the hardware ceiling**; 32×4 and 128×1 reach 91–95%, 4×32 83%. Transparent huge pages (512 MB on this system) matter only for the single-rank 1×128 layout, where first touch places whole arrays on one NUMA domain and the busiest controller group carries 1.66× the mean (26.9 s, against 13.3 s with THP disabled for the process); with four or more ranks every rank touches its own data and THP has no effect, so the baseline needs no change to the system.

Figure 1 places every kernel on the rooflines of one core and of the node. Every kernel lies at 0.13–0.83 flop/B, far left of the 5 flop/B ridge; even `viscosity` could reach only 17% of the FMA

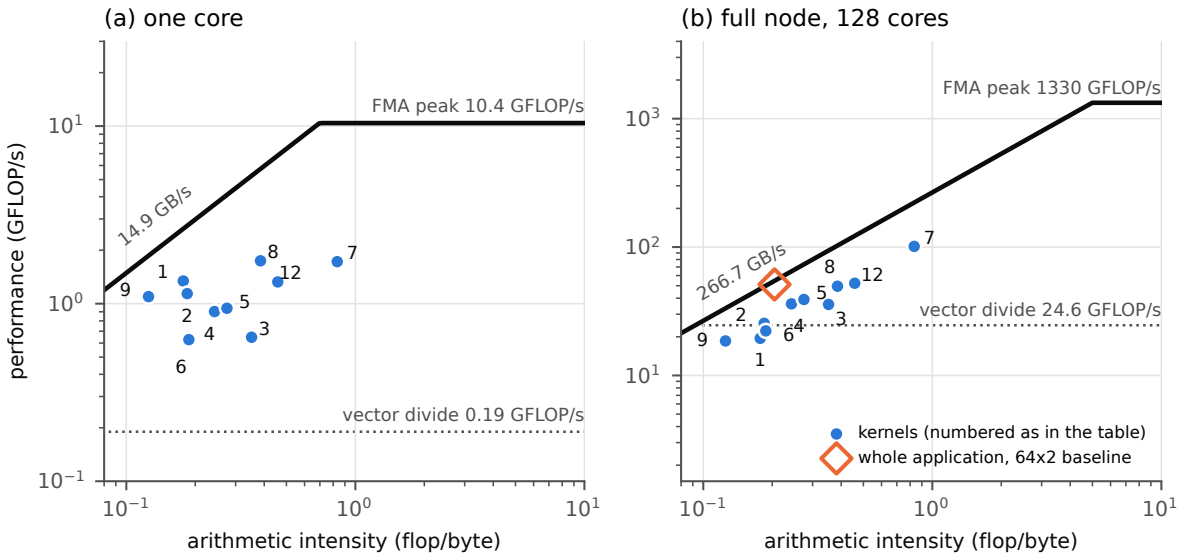


Figure 1: Roofline of the CloverLeaf C kernels (bm3840), one core and full node. Kernel numbers refer to Table 2; kernel points use model bytes and PAPI times (1×128), the application point the measured DRAM bytes and wall clock of the 64×2 baseline.

peak with perfect bandwidth. In hardware-byte terms (per-section roofline of the 64×2 run, `perf/roofline/roofline.py -hw`) both advection sections, PdV, `ideal_gas` and the fluxes run at 94–102% of the memory roof, acceleration at 83% and the scalar `sqrt` kernels at 63–66%. The floor at the measured traffic is 2.83 TB / 266.5 GB/s = **10.6 s**; the baseline is within 7% of it. There is no headroom at the current byte count: *every further gain has to come from moving fewer bytes*, and the Fortran build shows that this is possible (9.33 s, 2.36 TB, at the same 252 GB/s).

3 Kernel Improvements

What we have seen in the baseline analysis is that 94% of the memory bandwidth ceiling is already achieved by the base application. This means that we are highly memory bound, and the biggest performance gains will come from trying to reduce bytes. Only after this, instruction-side optimizations like vectorization could prove useful. We looked at the bytes the algorithm uses, and which can be removed / reused, as well as the bytes that are read from memory because of read-for-ownership, which we found to be present in the C kernels, but not the Fortran ones. We looked for opportunities to perform streaming stores there instead. In this section, we will explain which kernels we modified, in what way and what the expected gain from this improvement is. For every change, the correctness is verified by the correctness gate (explained in section 2.2) being passed.

The original plan was to take the three kernels with the largest theoretical gain in the byte model and leave the rest. These are: `advec_mom`, whose helper loops are recomputed for every velocity component. `reset_field` and `revert`, which solely copy data, and `advec_cell`, which round-trips four temporaries through memory. While reading these kernels, further opportunities presented themselves and were implemented as well: two dead stores (`post_vol`, `pre_vol`), the `volume_change` temporary of PdV, the fused reduction of `calc_dt`, two compiler flags that unblock vectorisation, `nowait` between loops that are provably independent and a branch-free

formulation of the `advec_mom` upwind loop. Changes (1)–(5) below reduce bytes, and (T1)–(T3) reduce time only. Section 3.4 details the implementation of each.

3.1 Flop and byte counts per cell

The counts of Table 3 were made by reading every loop nest of `kernels/*_kernel_c.c` and listing: the 2D arrays it reads, the 2D arrays it writes and the floating-point operations per cell. The bandwidth of bytes per cell of a loop nest is

$$8\,\mathrm{B} \times (\text{arrays read} + \text{arrays written} + \mathrm{WA} \times \text{arrays written but not read})$$

Our byte model counts as follows: Every array is streamed from DRAM once per loop nest, since *array_size* >> *cache*. Stencil neighbours $j \pm 1$, $k \pm 1$ hit in cache, and the read-for-ownership (WA) is charged once for every array a loop stores without loading. 1D arrays and halo cells are ignored. FLOPS: $+$, $-$, $\times$, $/$ and `sqrt` count one, and comparisons: `fabs`, `MIN`, `MAX` and `SIGN` count as zero. loop-invariant constants are not counted. For the kernels whose loops differ between the two sweeps, the average over the sweeps is taken. The DDR counters reported the same byte count result within 1% variability over 5 runs, for the 64x2 layout.

[TODO: For sergiu: move 3.1 to byte model explanation perhaps. I already edited this section and verified the claims]

3.2 Byte reductions

We show each of the 5 byte changes as comments on the baseline code snippets, so it can clearly be seen what the baseline code is doing, and what we will change. the new code follows in Section 3.4. In the C snippets `i` abbreviates the node-sized index `FTNREF2D(j,k,x_max+5,...)` of cell (j, k), `iy` the cell-sized one, and `i+1` the neighbour $j + 1$. `[(j,k-1)]` then means the niehgbour at coordinates $j, k - 1$.

(1) `advec_mom`: direction helper loops once per sweep. The kernel is called four times per step (two sweeps × `xvel`, `yvel`). Its first four loops depend only on the sweep direction, not on the velocity component, and are recomputed for `yvel` with identical results.

```
! advec_mom_driver.f90 (baseline): one kernel call per velocity component
CALL advec_mom_kernel_c(..., xvel1, mass_flux_x, vol_flux_x, mass_flux_y, vol_flux_y, &
                        volume, density1, work_array1..6, celldx, celldy,             &
                        which_vel=1, sweep_number, direction)
CALL advec_mom_kernel_c(..., yvel1, ... same arguments ...,                           &
                        which_vel=2, sweep_number, direction)
! Inside the kernel, loops 1-4 (post_vol, node_flux, node_mass_post, node_mass_pre)
! use only direction and sweep_number. When calling again for yvel, the same arrays
! are read again from memory and the same resulting arrays are computed. Only loop 5
! and 6 in this kernel read the velocity.
!
! Change: split the kernel in a preparation part called once per sweep and a velocity
! part called once per component.
```

(2) Temporaries in registers: `advec_cell` and PdV. The update loop of each `advec_cell` sweep writes four work arrays and reads them back in the same iteration, and nothing else reads them. Both PdV passes do the same with `volume_change`.

```
/* advec_cell_kernel_c.c (baseline), update loop of the x sweep */
pre_mass[i]  = density1[i]*pre_vol[i];                          /* stored here ...        */
post_mass[i] = pre_mass[i] + mass_flux_x[i] - mass_flux_x[i+1]; /* ... and reloaded here  */
post_ener[i] = (energy1[i]*pre_mass[i] + ener_flux[i] - ener_flux[i+1])
               / post_mass[i];
advec_vol[i] = pre_vol[i] + vol_flux_x[i] - vol_flux_x[i+1];
density1[i]  = post_mass[i] / advec_vol[i];
energy1[i]   = post_ener[i];
/* pre_mass, post_mass, post_ener and advec_vol are never read outside this iteration.
 * each costs a full-mesh store plus its read-for-ownership, four arrays of DRAM traffic
 * for values that fit in four registers.
 *
 * Change: make them scalars. */
```

```
/* PdV_kernel_c.c (baseline), predictor and corrector loops */
volume_change[i] = volume[iy]/(volume[iy]+total_flux);  /* stored here ...       */
...                                                      /* energy update         */
density1[iy]     = density0[iy]*volume_change[i];        /* ... and reloaded here */
/* volume_change (work_array1) is never read after PdV: calc_dt, advec_cell and
 * advec_mom write work_array1 before they read it. Per pass this is one full-mesh store
 * plus its read-for-ownership, 32 B per cell and step, for a value that fits in a register.
 *
 * Change: make it a scalar, as the Fortran PdV kernel already does. */
```

(3) Dead stores. `post_vol` in the first sweep of `advec_cell` and `pre_vol` in `advec_mom` are written over the whole haloed grid and never read.

```
/* advec_cell_kernel_c.c (baseline), sweep 1 (x direction) */
pre_vol[i]  = volume[i] + (vol_flux_x[i+1]-vol_flux_x[i])
                        + (vol_flux_y[iy+1]-vol_flux_y[iy]);
post_vol[i] = pre_vol[i] - (vol_flux_x[i+1]-vol_flux_x[i]);   /* post_vol never read */
/* advec_mom_kernel_c.c (baseline), every sweep */
post_vol[i] = volume[i] + vol_flux_y[iy+1] - vol_flux_y[iy];
pre_vol[i]  = post_vol[i] + vol_flux_x[i+1] - vol_flux_x[i];  /* pre_vol never read. The
                                                                 original source keeps it
                                                                 "for fidelity with the
                                                                 Fortran method" */
/* Change: delete both lines and the pre_vol argument. */
```

(4) Reset and Revert without copies. `reset_field` copies the four end-of-step fields (`density1`, `energy1`, `xvel1`, `yvel1`) into their level-0 partners, 118 MB each. `revert` copies `density0`/`energy0` back into the level-1 arrays before the PdV corrector because the predictor wrote into them.

```
/* reset_field_kernel_c.c (baseline): one of four identical copy loops */
#pragma omp for private(j)
for (k=y_min;k<=y_max;k++)
#pragma omp simd
  for (j=x_min;j<=x_max;j++)
    density0[i] = density1[i];  /* 118 MB read, 118 MB written, 118 MB read-for-ownership;
                                   the same for energy, xvel and yvel. No kernel needs a
                                   field at a particular address: the fields are
                                   allocatable components of the tile.

                                   Change: swap the buffers instead of copying the data. */
```

```
! PdV.f90 (baseline), end of the predictor pass
IF ( predict ) THEN
  CALL revert()
ENDIF
! density1 = density0, energy1 = energy0 over the whole tile, because
! the predictor wrote density1/energy1. The C corrector recomputes both
! from the level-0 fields over the same range and nothing in between
! reads the level-1 values.
!
! Change: skip the call on the C path.
```

(5) `calc_dt`: fused reduction. The kernel stores the local time step of every cell to `dt_min` and reduces the array in a second loop.

```
/* calc_dt_kernel_c.c (baseline) */
#pragma omp for private(...)
for (k...) for (j...) {
    ... the four limits dtct, dtut, dtvt, dtdivt of cell (j,k) ...
    dt_min[i] = MIN(dtct, MIN(dtut, MIN(dtvt, dtdivt)));  /* full-mesh store plus its
                                                             read-for-ownership */
}
#pragma omp for reduction(min:dt_min_val)
for (k...) for (j...)
    if (dt_min[i] < dt_min_val) dt_min_val = dt_min[i];    /* full-mesh reload of a value that
                                                             is only ever min-reduced */
/* The second pass exists only to separate the arithmetic from the reduction.
 *
 * Change: reduce inside the first loop, drop dt_min, locate the cell afterwards
 * and only when the step is too small. */
```

3.3 Time-only changes

After the amount of bytes moved is reduced, the following four items increase the AI, to move the kernel off the memory roof.

(T1) `nowait` between independent worksharing loops. From the baseline analysis we saw that 12.5% of the cycles of a 128-thread run are barrier time (Section 2.3, Table 2). Inspecting the kernels showed that many consecutive loops of a kernel touch disjoint arrays, or the same rows under the same static schedule.

(T2) `-fno-math-errno`: `calc_dt`, `ideal_gas` and `viscosity` run entirely scalar because GCC keeps `sqrt` as a library call that may set `errno`.

(T3) a vectorisable upwind loop: the advection kernels choose the upwind, donor and downwind indices by a branch on the sign of the flux and then load through those indices. Loading all candidates and selecting the value removes the branch, and GCC additionally needs `-fno-trapping-math` to evaluate the limiter's divisions speculatively under the select.

(T2) and (T3): compile commands only

```
# before: the project's GNU flags for the C kernels (Makefile, CFLAGS_GNU and OMP_GNU)
mpicc -O3 -march=native -funroll-loops -fopenmp -c kernels/*_kernel_c.c
# after: two flags added to CFLAGS_GNU (and CFLAGS_ARM)
mpicc -O3 -march=native -funroll-loops -fno-math-errno -fno-trapping-math -fopenmp \
      -c kernels/*_kernel_c.c
# Neither flag changes rounding or the order of operations, and CloverLeaf enables no
# floating-point traps, which means results should be identical. We later found that
# -fno-math-errno replaces the sqrt library call with a native sqrt, which does not
# always conform to strict accuracy boundaries. This caused FMA instructions to be
# fused in a different way than the library call, and resulted in different rounding.
# More about this in the results section.
```

3.4 Implementation status

All five byte changes are implemented, and so are (T1) and (T3) in `advec_mom`. In `advec_cell`, (T1) and (T3) are still open. (T2) was implemented, but withdrawn again because it changes the result of the correctness gate (Section 4). The PdV part of (2) was found while implementing the other changes, and was added last.

The work distributed over the group members and done in parallel: `advec_mom` with `calc_dt` and the compiler flags on one, `advec_cell` with Reset and Revert on the other. To see what every change is worth on its own, we measured every build separately: the baseline, the flags alone, each line on its own and the merge in one job on one node at 64×2, with five repetitions under the DDR-controller counters (Table 5).

The flags alone change neither bytes nor time (+0.7% and −0.2%). The `advec_cell` line removes 12.1% of the traffic and 12.3% of the time, the `advec_mom` line 14.9% and 15.7%. Merged, they remove 27.8% of the bytes (2.828 to 2.043 TB), against the 29.3% the byte model predicted. The wall clock follows the bytes (−27.0%, 11.31 to 8.25 s), because every build sustains 248–252 GB/s.

Two more builds follow from this. `optnf` is the merge with the baseline compiler flags, because the flags fail the correctness gate (Section 4). It takes 8.56 s and moves 2.052 TB. `pdv` is `optnf` plus the PdV part of (2). Against a baseline rerun in its own job (11.22 s) it takes 8.25 s and moves 2.012 TB: −28.8% bytes and −26.5% time, against the −30.8% the model predicts for all five changes together. `pdv` is the build we report in Section 4. Below we show the new code of every change; the comments explain what was done.

(1) `advec_mom` split. The kernel is split into a preparation part, called once per sweep, and a velocity part, called once per component.

```
! advection.f90: the direction-only part runs once per sweep and tile, before the
! two per-component calls
DO tile=1,tiles_per_chunk
  CALL advec_mom_prep_driver(tile,direction,sweep_number) ! C only: advec_mom_prep_c builds
                                                          ! node_flux, node_mass_post,
                                                          ! node_mass_pre and post_vol
                                                          ! (work_array1/2/3/6)
  CALL advec_mom_driver(tile,xvel,direction,sweep_number) ! advec_mom_vel_c(xvel1, ...)
  CALL advec_mom_driver(tile,yvel,direction,sweep_number) ! advec_mom_vel_c(yvel1, ...)
ENDDO
! advec_mom_vel_c receives only vel1, the three node arrays, mom_flux (work_array4),
! celldx, celldy and direction; mass_flux, vol_flux, volume, density1 and pre_vol
! (work_array5) are no longer passed. Nothing between the three calls touches the
! work arrays. The Fortran kernel is untouched: the prep driver does nothing for
! use_fortran_kernels.
```

(2) Temporaries in registers. The four work arrays of the `advec_cell` update loop and `volume_change` in PdV are now scalars inside the loop.

```
/* advec_cell_kernel_c.c, update loop of the x sweep (y sweep identical with the y fluxes) */
#pragma omp for private(j)
for (k=y_min;k<=y_max;k++) {
#pragma omp simd
  for (j=x_min;j<=x_max;j++) {
    /* the four temporaries live in registers: no store, no read-for-ownership */
    double pre_mass_s  = density1[i]*pre_vol[i];
    double post_mass_s = pre_mass_s + mass_flux_x[i] - mass_flux_x[i+1];
    double post_ener_s = (energy1[i]*pre_mass_s + ener_flux[i] - ener_flux[i+1]) / post_mass_s;
    double advec_vol_s = pre_vol[i] + vol_flux_x[i] - vol_flux_x[i+1];
    density1[i] = post_mass_s / advec_vol_s;   /* same operations in the same order as  */
    energy1[i]  = post_ener_s;                 /* before: the results are bit-identical */
  }
}
/* The pre_mass, post_mass, advec_vol and post_ener arguments stay in the Fortran
 * interface but are no longer touched. */
```

```
/* PdV_kernel_c.c, predictor and corrector loops */
const double volume_change_s = volume[iy]/(volume[iy]+total_flux);  /* in a register */
...                                                                 /* energy update */
density1[iy] = density0[iy]*volume_change_s;
/* Same operations in the same order: bit-identical results. The volume_change argument
 * stays in the Fortran interface but is no longer touched. */
```

(3) Dead stores. Both stores are deleted, and `pre_vol` is removed from the arguments of `advec_mom`.

```
/* advec_cell_kernel_c.c, sweep 1: only pre_vol is computed, the post_vol store is gone */
pre_vol[i] = volume[i] + (vol_flux_x[i+1]-vol_flux_x[i]) + (vol_flux_y[iy+1]-vol_flux_y[iy]);

/* advec_mom_kernel_c.c, advec_mom_prep_c: post_vol only; pre_vol and its argument are
 * gone. IDXN/IDXC are the node- and cell-sized FTNREF2D index macros of the file. */
#pragma omp for private(j) nowait   /* T1: the node_flux loop that follows does not read
                                       post_vol; the barrier after node_flux protects
                                       node_mass_post, the first reader (k-1 stencil) */
for (k=y_min-2;k<=y_max+2;k++) {
#pragma omp simd
  for (j=x_min-2;j<=x_max+2;j++)
    post_vol[IDXN(j,k)] = volume[IDXC(j,k)] + vol_flux_y[IDXC(j,k+1)] - vol_flux_y[IDXC(j,k)];
}
```

(4) Reset and Revert. `reset_field` now swaps the allocations of the level-0 and level-1 fields instead of copying them, and the C path no longer calls `revert`.

```
! reset_field.f90, C path: the level-1 fields become the level-0 fields by exchanging
! the allocations; reset_field_kernel_c is no longer called
DO tile=1,tiles_per_chunk
  CALL swap_field(chunk%tiles(tile)%field%density0, chunk%tiles(tile)%field%density1)
  CALL swap_field(chunk%tiles(tile)%field%energy0,  chunk%tiles(tile)%field%energy1)
  CALL swap_field(chunk%tiles(tile)%field%xvel0,    chunk%tiles(tile)%field%xvel1)
  CALL swap_field(chunk%tiles(tile)%field%yvel0,    chunk%tiles(tile)%field%yvel1)
ENDDO

SUBROUTINE swap_field(a,b)              ! three MOVE_ALLOCs: O(1), no memory traffic
  REAL(KIND=8), ALLOCATABLE, INTENT(INOUT) :: a(:,:), b(:,:)
  REAL(KIND=8), ALLOCATABLE :: t(:,:)
  CALL MOVE_ALLOC(a,t); CALL MOVE_ALLOC(b,a); CALL MOVE_ALLOC(t,b)
END SUBROUTINE swap_field

! PdV.f90: the C corrector rewrites density1/energy1 over the whole range revert would
! copy, so the C path skips the copy; the Fortran kernels keep it
IF ( predict .AND. .NOT. use_C_kernels ) THEN
  CALL revert()
ENDIF
```

(5) `calc_dt` fused reduction. The minimum is now reduced in the loop that computes it. The cell of the minimum is only searched for when the time step is too small.

```
/* calc_dt_kernel_c.c: one pass; dt_min is never written (the argument stays for the
 * Fortran interface). The four limits moved to an always_inline helper so that the hot
 * loop and the diagnostic scan share one definition. */
#pragma omp for private(j,dtct,dtut,dtvt,dtdivt) reduction(min:dt_min_val)
for (k=y_min;k<=y_max;k++) {
#pragma omp simd private(dtct,dtut,dtvt,dtdivt) reduction(min:dt_min_val)
  for (j=x_min;j<=x_max;j++) {
    cell_dt_limits(j,k, ..., &dtct,&dtut,&dtvt,&dtdivt);  /* sound, x-vel, y-vel, divergence */
    dt_min_val = MIN(dt_min_val, MIN(dtct,MIN(dtut,MIN(dtvt,dtdivt))));
  }                    /* every thread and vector lane keeps a private minimum; min is
                          exact, so the order in which OpenMP combines them is irrelevant */
}
if (dt_min_val < min_dt) {        /* failure path only: serial, runs once before the abort. */
  for (k...) for (j...) {         /* Re-evaluate the limits and report the first cell that  */
    cell_dt_limits(...);          /* reaches the minimum and which of the four it was,      */
    if (MIN(dtct,MIN(dtut,MIN(dtvt,dtdivt))) == dt_min_val) {  /* instead of the fixed      */
      j_ldt=j; k_ldt=k; dtl_control=...; goto found;           /* (1,1)/"sound" of the port */
    }
  }
}
```

(T1) `nowait`: implemented in `advec_mom`, open in `advec_cell`. A barrier can only be removed when the next loop reads, from the arrays the first loop writes, only the rows the same thread wrote. OpenMP guarantees this for two loops with the same iteration range and `schedule(static)`. That holds along j in `advec_mom`, but not along k, where the stencil reads row $k \pm 1$ of another thread.

```
/* advec_mom_prep_c, direction 1: node_mass_post -> node_mass_pre without a barrier */
#pragma omp for private(j) schedule(static) nowait  /* same k range and schedule(static) as
                                                       the next loop: OpenMP guarantees the
                                                       same row -> thread mapping, and
                                                       node_mass_pre reads node_mass_post
                                                       only in its own row */
for (k=y_min;k<=y_max+1;k++)
#pragma omp simd
  for (j=x_min-1;j<=x_max+2;j++)
    node_mass_post[IDXN(j,k)] = 0.25*(density1[IDXC(j  ,k-1)]*post_vol[IDXN(j  ,k-1)]
                                     +density1[IDXC(j  ,k  )]*post_vol[IDXN(j  ,k  )]
                                     +density1[IDXC(j-1,k-1)]*post_vol[IDXN(j-1,k-1)]
                                     +density1[IDXC(j-1,k  )]*post_vol[IDXN(j-1,k  )]);
#pragma omp for private(j) schedule(static)         /* node_flux finished at its own barrier */
for (k=y_min;k<=y_max+1;k++)
#pragma omp simd
  for (j=x_min-1;j<=x_max+2;j++)
    node_mass_pre[IDXN(j,k)] = node_mass_post[IDXN(j,k)]
                             - node_flux[IDXN(j-1,k)] + node_flux[IDXN(j,k)];
/* Also nowait: the post_vol loop (see (3)) and, in direction 1 of advec_mom_vel_c, the
 * mom_flux loop before the velocity update (same rows, same schedule). Direction 2
 * keeps its barriers because the stencils cross rows. advec_cell is still all-barrier. */
```

(T2) Compiler flags: implemented and withdrawn. The two flags were added to the GNU and ARM flag sets of the Makefile, as shown in Section 3.3. They change the result of the correctness gate, so the reported build uses the baseline flags again (Section 4).

(T3) Branch-free upwind loop: implemented in `advec_mom`, open in `advec_cell`. The upwind, donor and downwind values are now selected from four contiguous loads, instead of loaded through indices chosen by a branch. The vector form of this loop needs `-fno-trapping-math` (Section 3.3), so it only pays off together with (T2).

```
/* advec_mom_vel_c, mom_flux loop, direction 1 (direction 2 is the same along k with celldy) */
#pragma omp for private(j) schedule(static) nowait
for (k=y_min;k<=y_max+1;k++) {
#pragma omp simd
  for (j=x_min-1;j<=x_max+1;j++) {
    const double nf  = node_flux[IDXN(j,k)];
    const int    neg = (nf < 0.0);   /* flow in -j: upwind j+2, donor j+1, downwind j */
    const double v_m1 = vel1[IDXN(j-1,k)], v_0  = vel1[IDXN(j  ,k)],  /* all four candidates: */
                 v_p1 = vel1[IDXN(j+1,k)], v_p2 = vel1[IDXN(j+2,k)];  /* contiguous loads     */
    const double v_up  = neg ? v_p2 : v_m1;      /* selects replace the branch and the      */
    const double v_don = neg ? v_p1 : v_0;       /* indirect loads through upwind/donor/    */
    const double v_dwn = neg ? v_0  : v_p1;      /* downwind                                */
    const double m_don = neg ? node_mass_pre[IDXN(j+1,k)] : node_mass_pre[IDXN(j,k)];
    const double width = celldx[FTNREF1D(j,x_min-2)];
    const double w_dif = neg ? celldx[FTNREF1D(j+1,x_min-2)] : celldx[FTNREF1D(j-1,x_min-2)];
    const double sigma   = fabs(nf)/m_don;
    const double vdiffuw = v_don - v_up,  vdiffdw = v_dwn - v_don;
    const double auw = fabs(vdiffuw), adw = fabs(vdiffdw);
    const double wind = (vdiffdw <= 0.0) ? -1.0 : 1.0;
    const double lim  = wind*MIN(width*((2.0-sigma)*adw/width+(1.0+sigma)*auw/w_dif)/6.0,
                                 MIN(auw,adw));  /* always evaluated (-fno-trapping-math) */
    const double limiter = (vdiffuw*vdiffdw > 0.0) ? lim : 0.0;  /* ... then selected */
    mom_flux[IDXN(j,k)] = (v_don + (1.0-sigma)*limiter)*nf;
  }
}
/* Same arithmetic in the same order as the branchy original: bit-identical results.
 * With -fno-trapping-math GCC vectorises all 13 loops of the file (vectoriser report,
 * GCC 13, x86); the NEON share on the cluster is measured with PAPI. */
```

build	changes	wall (s) median (min–max)	DRAM (TB)	bytes vs base (%)	time vs base (%)
base	M3 baseline source and CFLAGS	11.31 (11.24–11.33)	2.828	+0.0	+0.0
flags	T2 only: -fno-math-errno -fno-trapping-math (no kernel change)	11.28 (11.27–11.34)	2.846	+0.7	−0.2
cell	flags + advec_cell register temporaries, dead post_vol store, reset/revert by field swap	9.92 (9.84–9.94)	2.486	−12.1	−12.3
mom	flags + advec_mom helper hoist, dead pre_vol store, calc_dt fused reduction (not cell)	9.54 (9.54–9.62)	2.405	−14.9	−15.7
opt	all changes (merge of cell and mom)	8.25 (8.23–8.33)	2.043	−27.8	−27.0
optnf	opt with the baseline CFLAGS (no -fno-math-errno -fno-trapping-math)	8.56 (8.53–8.63)	2.052	−27.4	−24.3
pdv	optnf + PdV volume_change kept in a register	8.25 (8.25–8.37)	2.012	−28.8	−26.5

Table 5: Per-change attribution at 64×2 (bm3840, five repetitions under the DDR-controller counters, one node, one job): the ladder of builds, each against the baseline build of the same job (the `optnf` and `pdv` rows were measured later, each against a baseline rerun of its own, 11.32 and 11.22 s). Filled from `data/ladder.csv` (`perf/jobs/m3_opt.sbatch`).

4 Results

Every optimised build is measured exactly like the baseline (Section 2.2): five repetitions under the DDR-controller counters at the same five layouts, the correctness gate on `clover_bm.in` and `PASSED` on bm3840. We first discuss correctness, because it decides which build we report, and then the performance of that build.

Correctness. We ran the `clover_bm.in` gate for every build of the ladder, and for the ladder's own baseline binary (Table 6). The baseline build reproduces the archived C deviation of 0.0057% at every layout and with THP off, and it reproduces the Fortran value as well. This means the reference stands, now from a build of which we know the source and the flags. The four C builds with the compiler flags, the flags-only build included, all report a different deviation of 0.038% at every layout. The Fortran path of the optimised build reproduces its reference to all digits. The kernel changes are therefore neutral, and the change of the result is caused by `-fno-math-errno -fno-trapping-math` alone: the flags-only build differs from the baseline only in these two flags and in comments.

The mechanism is the fused multiply-add contraction. Sergiu's bisect (`perf/jobs/m3_flagbisect.sbatch`) ran three source revisions with three flag sets each. It shows that with `-ffp-contract=off` the original kernels, the `advec_cell` edits and the merged tip all print the same 2×10^{-13}%. So the kernel edits are arithmetically identical to the originals. Both 0.0057% and 0.038% are what GCC's default `-ffp-contract=fast` makes of the code: the flags vectorise the `sqrt` kernels and if-convert the upwind loops, and the vector code fuses different products. Both values are physically irrelevant and both fail CloverLeaf's own 0.001% threshold. Our gate, however, is reproduction to 10^{-6}% (Section 2.2), and the build with the flags does not pass it.

We therefore dropped the flags. `optnf`, the five byte changes with the baseline flags, reproduces 0.0057% at every layout and with THP off, and the Fortran value (7 of 7 runs). `pdv`, which is `optnf` with the PdV part of (2), does the same (7 of 7 runs); it is the build we report below. All

kernels	build	runs	deviation (%)	layouts (all identical)	gate
c	base	22	0.5726209E-02	64x2, 128x1, 32x4, 4x32, 1x128	ok
fortran	base	12	0.1787088E-01	64x2, 128x1, 4x32	ok
c	opt	6	0.3835202E-01	64x2, 128x1, 32x4, 4x32, 1x128	MISMATCH
fortran	opt	1	0.1787088E-01	64x2	ok
c	flags	1	0.3835202E-01	64x2	MISMATCH
c	cell	1	0.3835202E-01	64x2	MISMATCH
c	mom	1	0.3835202E-01	64x2	MISMATCH
c	optnf	6	0.5726209E-02	64x2, 128x1, 32x4, 4x32, 1x128	ok
fortran	optnf	1	0.1787088E-01	64x2	ok
c	pdv	6	0.5726209E-02	64x2, 128x1, 32x4, 4x32, 1x128	ok
fortran	pdv	1	0.1787088E-01	64x2	ok

Table 6: Kinetic-energy deviation of `clover_bm.in` per build (names as in Table 5; `opt` is the merge with the compiler flags of (T2)). The `base` rows are the values an optimised build has to reproduce; the gate column is the verdict of `qa_check.py` against them.

layout	baseline			optimised			speed-up
	wall (s)	TB	GB/s	wall (s)	TB	GB/s	
64x2	11.32 (11.24–11.33)	2.83	250	8.26 (8.25–8.34)	2.01	244	1.37
1x128	26.91 (17.44–29.20)	2.99	111	20.25 (15.04–22.74)	2.09	103	1.33
4x32	13.00 (12.76–13.21)	2.87	221	9.46 (9.29–9.72)	2.00	212	1.37
32x4	11.46 (11.41–11.47)	2.79	244	8.43 (8.42–8.51)	1.98	235	1.36
128x1	11.66 (11.63–11.77)	2.95	253	8.54 (8.54–8.55)	2.11	247	1.36

Table 7: C kernels, bm3840: baseline against the optimised build `pdv` at every layout, THP as installed. Speed-up = baseline median / optimised median.

builds print `PASSED` on bm3840.

What the flags were worth. On the baseline the flags are worth nothing (+0.7% bytes and −0.2% time, Table 5). On the optimised kernels they are worth 3.6% of the wall clock (8.25 against 8.56 s at the same bytes). Almost all of it is in the two `sqrt` kernels, whose vectorised square root only shows once the bytes around them are reduced: `calc_dt` takes 0.45 against 0.62 s and `ideal_gas` 0.45 against 0.54 s. That is the price of reproducing the reference, and the PdV part of (2) happens to buy exactly that back: 8.25 s with the flags but without the PdV part, 8.25 s without the flags but with it. We pay this price, because the gate is this project's definition of correct. The flags stay available as a separately measured option for a user with a looser criterion.

Performance. Table 7 compares every layout with its own baseline, and Figure 2 shows the wall clock against the memory floor. The gain comes from the byte reduction alone. Every layout loses 28–30% of its traffic and speeds up by about the same factor: 1.36–1.37× at the four balanced layouts and 1.33× at the NUMA-bound 1×128. The THP-on wall clock of 1×128 scatters by a third between repetitions; with THP off its speed-up is 1.35×. The ladder adds up change by change (Table 5), and the counters confirm the byte model of Section 3.2 to within two percentage points (−28.8% measured against −30.8% predicted). The optimised C build is now faster than the Fortran reference at every layout (Figure 2). This means the C/Fortran byte gap of Section 2.5 is closed and reversed.

What did not change. The character of the code is the same as before. Figure 3 shows every kernel still on the bandwidth roof, with IPC and back-end stall share as before (Table 8). The whole application moved up along the roof, not off it. It runs at 91% of the hardware ceiling (244 GB/s), with 9% of the wall clock left above the memory floor of its own byte count (7.55 s).

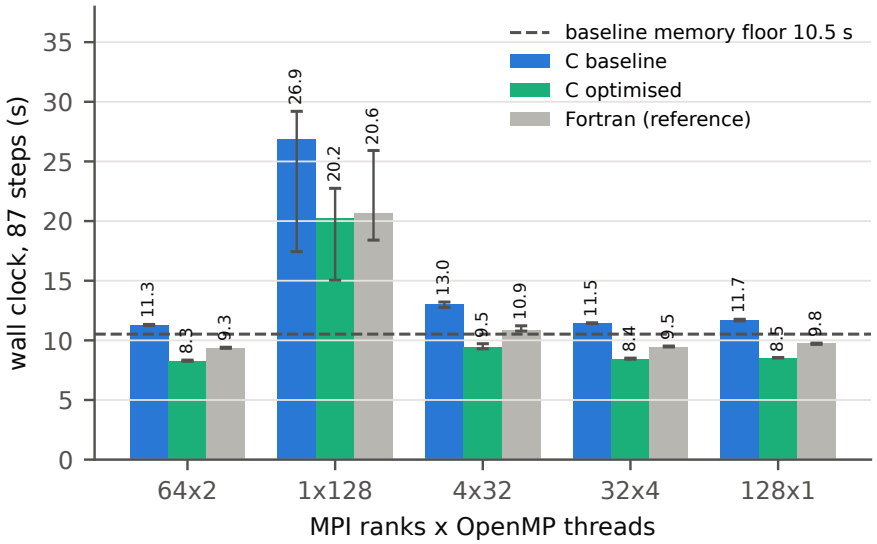


Figure 2: Wall clock per layout (median, bars min–max): C baseline, C optimised where measured, Fortran for reference. The dashed line is the memory floor of the baseline byte count at 266.5 GB/s.

Per kernel, the two advection kernels took most of the gain (1.60× and 1.45×), and Reset and Revert vanished. The PdV part of (2) is worth 1.13× on both PdV passes: 40 GB less, which is 1.4% of the baseline bytes against the model's 1.5%. `calc_dt` and `ideal_gas` gain 5–6% without their vectorised square root. `accelerate`, `flux_calc` and `viscosity` are untouched, and they are the next bytes to remove. The time-only items (T1 and T3 in `advec_cell`, and the withdrawn flags) cannot buy more than the remaining 9% at this layout. Only fewer bytes can, for example by removing the read-for-ownership with streaming stores (Section 5.1).

5 Conclusions (draft)

- The C kernels are memory-bound (0.2 flop/B against a 5 flop/B ridge) and the 64×2 baseline already sustains 94% of the 266.5 GB/s hardware ceiling over the whole run: only fewer bytes can make it faster.
- The analytic byte model matches the counters to 1% and predicts the five planned changes at −29% of the traffic (floor 10.5 to 7.4 s); the read-for-ownership that the C stores pay, and that streaming stores and the Fortran build avoid, is a sixth lever worth up to another −23%.
- Measured: the five byte changes remove 27.8% of the DRAM traffic (model: 29.3%) and the wall clock follows, 1.37× at 64×2 and 1.31–1.39× at every other layout; the optimised C build is faster than the Fortran reference everywhere and still sits on the bandwidth roof, 7% above the memory floor of its own bytes.
- The correctness gate: every build of the ladder, flags-only included, reproduces one and the same `clover_bm.in` deviation at every layout and the Fortran path its reference; the archived C reference value is reproduced by no build that can be made from the sources, and is being re-established from the rebuilt baseline binary.

kernel	time (s)			IPC		back-end stall (%)	
	base	pdv	ratio	base	pdv	base	pdv
advec_mom	4.19	2.62	1.60	0.27	0.28	91	89
advec_cell	2.42	1.67	1.45	0.28	0.36	91	88
pdv_correct	0.71	0.63	1.13	0.20	0.23	92	92
calc_dt	0.66	0.62	1.05	0.50	0.55	81	80
pdv_predict	0.61	0.54	1.13	0.22	0.24	92	90
ideal_gas	0.58	0.54	1.06	0.31	0.32	87	87
accelerate	0.56	0.61	0.93	0.27	0.26	91	91
flux_calc	0.39	0.39	1.01	0.16	0.16	95	95
reset_field	0.38	–	–	0.10	–	96	–
viscosity	0.32	0.33	0.98	0.99	0.96	62	63
revert	0.19	–	–	0.10	–	96	–
update_halo	0.04	0.04	1.05	0.57	0.60	42	41
field_summary	0.03	0.03	1.03	0.42	0.43	85	85

Table 8: Per-kernel PAPI counters at 64×2, THP on: baseline against the optimised build `pdv`. Time is the region time of the ipc event set (max over threads); Reset and Revert have no C region any more (field swap). The two halves of the split `advec_mom` count under one region.

5.1 Future Work

We have identified during the baseline performance analysis, that the WA attributes to a significant portion of bytes moved in the C kernels. Due to lack of time we have not been able to implement and benchmark the improvement of rewriting these stores into explicit streamed stores, but this is something that we hypothesise will result in a significant performance improvement.

References

[1] Jan Laukemann, Georg Hager, Gerhard Wellein, et al. CloverLeaf on Intel multi-core CPUs: A case study in write-allocate evasion. arXiv:2311.04797, 2023.

[2] A. C. Mallinson, D. A. Beckingsale, W. P. Gaudin, J. A. Herdman, J. M. Levesque, and S. A. Jarvis. CloverLeaf: Preparing hydrodynamics codes for exascale. In *Proceedings of the Cray User Group (CUG)*, 2013.

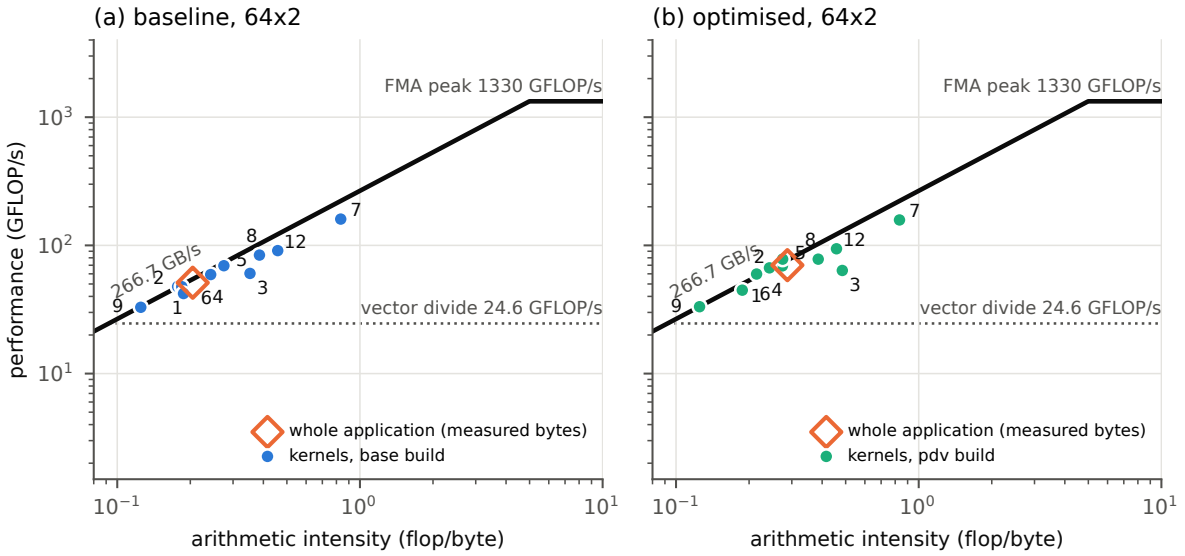


Figure 3: Node roofline at 64×2, baseline (a) against the optimised build (b): kernels from their PAPI region times and the byte model of each build, numbered as in Table 2; the hollow diamond is the whole application from the measured DRAM bytes and wall clock.